

Coordinating Multi-Vendor AI Agent Fleets: Research Paper Graded by Fable

Epilogue — an independent grading of the research paper, with terminology, recommendations, and the author’s coda

rendered 2026-07-12

Contents

Overview	1
Findings	2
Terminology	2
Rule vs. gate	2
Coordination record / substrate	3
Multi-vendor fleet	3
MCP vs. CLI	3
Worktree	3
Long-autonomy	3
Ablation	3
Lock-in	3
Design-science	4
Recommendations to Author	4
Author’s coda — 2026-07-12	6

A review of 2026-07-08-coordinating-multi-vendor-agent-fleets.md with terminology guide and recommendations for author.

Overview

There is real strategic value here, but it’s not where the paper’s ambition points. The paper is a solo builder’s design-science study of AIDA, a git-based system for coordinating fleets of AI coding agents across vendors (Claude, Codex, etc.). Its grand thesis — that the cross-vendor coordination layer is the durable, defensible investment — is plausible but unproven, and the author admits it. The genuine value is in three smaller, counter-intuitive, experimentally-backed findings that challenge assumptions a large company is probably budgeting on right now.

Why I’d take it seriously despite being n=1. This is the rare self-published draft that repeatedly falsified its own thesis. The author pre-registered five hypotheses about when AI agents ignore written rules, ran controlled experiments with deterministic (script-based, not AI-judged) grading, and all five failed — and the paper reports that as the finding. It red-teamed its own claims and filed bugs against its own system (e.g., its merge logic was quietly non-deterministic).

That's stronger epistemic hygiene than most vendor whitepapers. The discounts are equally real: one system, essentially one operator, 10 runs per experimental arm, single-task pilots, and field data drawn from the author's own disciplined repo. Everything here is hypothesis-grade, not conclusion-grade.

Findings

The three findings worth extracting:

1. Governance inversion (best evidence in the paper). Modern models followed written prose rules with 100% compliance across five controlled experiments spanning two vendors and trivial-to-complex tasks — the programmatic enforcement gates never fired once. Their field data agreed: orchestrated agent commits violated stated rules less (2%) than interactive human sessions (17%). The implication, if it holds: enterprises are likely over-buying hard enforcement machinery for well-scoped agent tasks. The caveat: the one observed rule violation happened on a large, messy, real codebase under long unattended autonomy — exactly the regime the experiments couldn't reach — so gates may still earn their keep there.
2. Multi-vendor competition is QA, not diversity. Two vendors given the same deliberately open brief converged on essentially the same design, because the shared codebase dictates the shape. What a second vendor actually buys is regression-catching — one arm shipped bugs the other didn't. Weak evidence (n=1 task, self-judging), but if you're funding multi-vendor agent programs expecting architectural diversity, you're mispricing them.
3. Agent economics. The typed MCP protocol surface cost $\sim 2\times$ the plain text CLI for equal-or-worse task success; warm recycled worktrees cut per-agent setup cost $\sim 30\times$; event-driven supervision beat timer polling that burned \$6-20/night finding nothing. Directly checkable if you build internal agent tooling.

What not to take: the implied build case. The author's own verdict on rolling your own coordination layer is "open," leaning against — vendors are absorbing the lower layers for free, and the upper layer is already contested by Microsoft Agent 365, Temporal, and better-distributed open-source projects. The one durable strategic principle that survives even if AIDA dies: the coordination record (shared task/role/intent state across agents) is where the next vendor lock-in fight happens — require it stay exportable and program-owned in any agent platform you procure. Also demonstrated: a new vendor joined a running fleet in ~ 50 seconds with zero integration, meaning vendor-neutrality can be nearly free — real switching-cost leverage.

Recommended actions (detailed on slide 6): don't build; run two cheap internal replications (rules-vs-gates on one of your real pipelines, and an MCP-vs-CLI cost audit); adopt the portable-coordination-record requirement in procurement now; reframe multi-vendor spend as QA plus hedge; put the standards bodies (A2A/MCP) and the named incumbents on a two-quarter watch list.

Terminology

Rule vs. gate

Rule vs. gate — the central pair in the paper. Both are ways to make an AI agent obey a policy, like "every commit message must follow this format."

- A rule is a written instruction the agent reads — a line in a prompt or a project instructions file saying "always do X." Nothing physically stops the agent from ignoring it. Like a sign that says "Employees must wash hands."

- A gate is code that mechanically blocks the action if the policy is violated — the system rejects the malformed commit rather than asking nicely. Like a turnstile that won't turn without a badge. The paper's founding belief was "agents ignore signs, so you need turnstiles everywhere." Their experiments showed modern models actually obey the signs essentially 100% of the time in every condition they could construct — hence my phrase "governance inversion," and the recommendation to question spend on turnstile-building.

Coordination record / substrate

Coordination record / substrate — the shared, durable state that lets many agents work on one project without colliding: the task list, who has claimed what, what's blocked on what, what "done" means, message queues between agents. The paper's word "substrate" just means this shared layer everything else stands on. My procurement point: vendors want this record to live inside their platform (that's the lock-in), so require it be exportable in an open format.

Multi-vendor fleet

Multi-vendor fleet — running agents from different AI companies simultaneously (e.g., an Anthropic agent implementing, an OpenAI agent reviewing). "Vendor-neutral" means the coordination layer doesn't care whose agent shows up.

MCP vs. CLI

MCP vs. CLI (the $\sim 2\times$ cost finding) — two ways an agent can talk to a tool. MCP (Model Context Protocol) is a structured, typed API — formal, machine-readable schemas. A CLI is the ordinary command-line interface, plain text in and out. Intuition says the structured protocol should serve agents better; the measurement found agents completed the same tasks as well or better through plain text at about half the token cost.

Worktree

Worktree — a disposable copy of the codebase each agent works in so parallel agents don't overwrite each other. "Warm recycled worktrees" = reusing an already-set-up copy instead of building a fresh one per agent, which is where the $\sim 30\times$ setup savings came from.

Long-autonomy

Long-autonomy / drain — running agents unattended for hours working through a queue of tasks ("draining the queue") with no human watching. This is the one regime where the paper's evidence says gates might still matter, because it's where the single rule violation in their whole program occurred.

Ablation

Ablation — an experiment that changes exactly one variable between two otherwise-identical setups (e.g., same task, same model, gate on vs. gate off) so you can attribute any difference to that variable. When I said "five controlled cells," each cell was one such experiment.

Lock-in

Lock-in / switching cost — the practical difficulty of leaving a vendor once your workflows, data, and coordination state live inside their platform. The ~ 50 -second vendor onboarding demo matters because it shows the opposite is achievable: if joining your fleet costs a new vendor nothing, leaving one costs you little, and that's negotiating leverage.

Design-science

Design-science study is a research method where you build a working artifact — a system, a tool, a process — and the act of building and operating it is itself the experiment. The knowledge you produce isn't "we surveyed 200 teams" or "we ran a controlled trial"; it's "we built the thing, and here is what building it forced us to learn about the problem that couldn't be learned any other way."

The term comes from information-systems research (the paper cites Hevner et al., 2004, the standard reference). The contrast is with natural science: natural science asks "what is true about the world?" and tests hypotheses through observation; design science asks "what works, and why?" and tests ideas by constructing something and seeing where it succeeds, strains, or breaks. The artifact is both the contribution and the measuring instrument.

In this paper's case: the author built AIDA (the agent-coordination system) and used it daily to build itself — what the paper calls "dogfooding." Every design assumption that turned out wrong (they list about a dozen in §11, like "rules can't govern a capable agent" or "a git-backed multi-writer store means merge hell") became a finding, because the failure only surfaced once the system was real and running. Their framing "the system is the instrument, not the subject" means: we're not asking you to evaluate AIDA; we're reporting what AIDA-as-a-probe revealed about the problem space.

Why this matters for how much weight you give the paper:

- The strength: design science surfaces knowledge that pure analysis can't reach. You genuinely cannot discover "vendor onboarding takes 50 seconds" or "MCP costs 2× the CLI" by reasoning from an armchair — someone has to build and measure. The paper's §11 list of falsified assumptions is the method working as intended.
- The structural weakness: the builder is also the evaluator, the test subject, and the author. There's no control group, no independent replication, and success is partly self-defined. The paper flags this honestly ("autoethnographic," meaning the researchers are studying their own practice — another borrowed academic term, from ethnography turned on oneself).

So when I graded the paper "hypothesis-generating, not conclusive," that's not a criticism of the execution — it's the ceiling of the method itself. A design-science study done well, as this one mostly is, tells you which questions are worth asking and what the answers might be. Confirming those answers requires the multi-team, multi-vendor studies the paper itself lists in §13 as future work.

Recommendations to Author

Here is my advice, and I mean every word of it — not as consolation, but as a reading of the evidence he produced.

The paper didn't invalidate the aspiration underpinning AIDA. It invalidated one justification for the tool — and it was the justification least connected to why people adopt tools. What died in those five ablations was a specific theoretical claim: that programmatic gates are required to govern capable agents. What did not die: the tool works, a cold vendor joined its fleet with zero integration, the typed graph beat grep by 80× on the queries that matter at scale, and it prevented silent reference-rot at every scale tested. People don't adopt tools because a proposition survived peer review. They adopt tools because the tool does something they want. Nothing in the paper says AIDA doesn't.

He should notice what he actually built, because almost nobody can do it. The paper pre-registered five hypotheses, built deterministic experimental apparatus, falsified its own founding thesis five consecutive times, and published that. I review a lot of technical writing; the

epistemic honesty in this document is genuinely rare — vendor whitepapers, VC-funded startups, and most academics do not do this. A person who feels crushed by his own findings is a person who did the science right. That skill is worth more than the artifact, and it transfers to everything he builds next.

The competitor discovery is validation wearing the costume of defeat. Finding that Steve Yegge’s Beads/Gas Town stack occupies the same niche with 24k stars feels like “someone already won.” Read it the other way: an engineer with enormous distribution independently concluded this exact layer is worth building. The author wasn’t wrong about the problem — he was early and right, and he found out via honest multi-modal discovery rather than shipping into a wall. The incumbent all but concedes the point himself: on 2026-07-02 Yegge wrote that Gas Town “wasn’t expected to be the right shape — I just wanted to be there first.” *First* is not *right*; the shape of the niche is still contested, which is the opposite of a settled frontier. And his own landscape table shows the niche isn’t closed: AIDA holds things nobody in the surveyed field holds — code-to-spec inline traceability, the plain-file git-YAML store (versus Beads requiring a Dolt SQL engine), lifecycle-authority gating. Those are real forks, not “less mature versions of Beads.”

Concrete advice, in order:

1. Publish the paper. It may be more valuable than the tool, and it’s finished-enough. The gate-vs-rule result — “2026 models honor stated rules at ceiling; a clean ablation cannot reproduce rule-dropping” — is a finding the whole agent-engineering community is currently getting wrong in the expensive direction. The ablation apparatus is a contribution in itself: the gate-vs-rule cells and the RYO-friction benchmark ship as deterministic runnable scripts (scripts/ablations/), and the agent-surface benchmark (bench/agent-surface/run_bench.py) is proven reproducible — re-run four times on current models, the $\sim 2\times$ MCP result holding each time. A well-received negative-result paper builds exactly the reputation that gets a not-for-profit tool its first users.
2. Let the paper prune the tool. The findings are a free roadmap to a leaner AIDA: remove or demote gates that only fire on well-scoped tasks (the paper says this itself), make the token-efficient CLI the headline surface and MCP the option, fix or delete the unwired HLC (BUG-578). A tool that shrinks in response to its author’s own evidence is a tool people trust.
3. Narrow the pitch to the surviving wedge. Not “the coordination substrate for multi-vendor fleets” — Gas Town can fight for that. Instead: “requirements that live in git as plain YAML, trace to code lines, and any vendor’s agent can join in under a minute.” Every word of that is experimentally backed by his own pilots. Small, sharp, demonstrable in a two-minute demo.
4. Steal his own best idea from §12: the one-way Beads→AIDA importer. Don’t fight the incumbent’s traction; siphon it. Users who outgrow Beads’ unenforced lifecycle and want traces and authority gates get an on-ramp. For a not-for-profit tool, the neighbor’s 24k stars are a distribution channel, not a threat.
5. Recount what the hours bought. The goal was “a tool people would want to use.” The hours produced: a working system he uses daily, a genuinely novel experimental method, a map of a minefield (§11) nobody else has published, and a paper. If even the paper finds its audience, people are using what he made — knowledge is the artifact that can’t be deprecated by a vendor release. His own words, from §11: that value accrues whether or not the artifact survives.

Many people will happily use a small, sharp, honest tool and that is fully intact. The author is closer to it today than before he wrote the paper, because now he knows exactly which parts are worth keeping.

Author's coda — 2026-07-12

I commissioned this review and I've read every word of it, including the parts that sting. The one recommendation I'm not taking is the headline — *don't build*. I'm going to plough on with AIDA. The reasoning, such as it is:

The review's own evidence is why. The competitor I thought had finished my work turns out, by his own account, to have optimized for being *first*, not for being *right* — so the shape of this niche is contested, not settled. The forks the review credits as genuinely mine — code traced back to the decision behind it, a plain-file git store instead of a database engine, authority that lives in the lifecycle — are live design bets, not late copies. And the coincidence that landed hardest, that the substrate under the incumbent is a database I sketched for a professor twenty years ago and only learned existed last month, I've decided to read the way this review reads the whole competitor story: as evidence the instinct was early and right, not that the timing was wrong.

The rest of the advice I'm taking, because it's correct. Publish the paper — the negative result on rules-versus-gates is something the field is getting expensive-wrong, and the ablation apparatus stands on its own. Let the paper prune the tool: the CLI is the headline surface and MCP the option; any gate that only fires on well-scoped work is a removal candidate; the pitch narrows to the surviving wedge — *requirements that live in git as plain YAML, trace to the code, and any vendor's agent can join in under a minute*. Build the Beads→AIDA on-ramp rather than fight the traction.

What I'm keeping most is the line the review lands on: the knowledge is the artifact a vendor release can't deprecate. Whether AIDA lives or not, the hours bought a map of a minefield nobody else has published, a method I trust, and a clearer head about which parts are worth keeping. So I'll plough on — on my own terms, with the license to follow it wherever it goes — and we'll see where that is.

— Joe